MIPT & YSDA · AUTUMN 2026

Deep Generative Models

Lecture 12

Roman Isachenko

Moscow Institute of Physics and Technology
Yandex School of Data Analysis

Recap of Previous Lecture

Use the familiar additive-noise form: $\mathbf{x} = \mathbf{x}_0 + \sigma\boldsymbol{\epsilon}$, $\boldsymbol{\epsilon} \sim \mathcal{N}(0, \mathbf{I})$.

Choose a denoiser using Tweedie's formula

$$\mathbb{E}[\mathbf{x}_0 \mid \mathbf{x}] = \mathbf{x} + \sigma^2 \nabla_{\mathbf{x}} \log p_\sigma(\mathbf{x}).$$

$$D_{\boldsymbol{\theta}}(\mathbf{x}, \sigma) \approx \mathbb{E}[\mathbf{x}_0 \mid \mathbf{x}], \qquad \frac{d\mathbf{x}}{d\sigma} = -\sigma \mathbf{s}_{\boldsymbol{\theta}}(\mathbf{x}, \sigma) = \frac{\mathbf{x} - D_{\boldsymbol{\theta}}(\mathbf{x}, \sigma)}{\sigma}.$$

Decreasing σ moves towards the denoiser prediction.

Train the Denoiser

$$\begin{aligned} \mathcal{L} &= \left\| \frac{D_{\boldsymbol{\theta}}(\mathbf{x}, \sigma) - \mathbf{x}}{\sigma^2} + \frac{\mathbf{x} - \mathbf{x}_0}{\sigma^2} \right\|_2^2 \\ &= \frac{1}{\sigma^4} \left\| D_{\boldsymbol{\theta}}(\mathbf{x}, \sigma) - \mathbf{x}_0 \right\|_2^2 . \end{aligned}$$

Preserving the score objective requires the factor $1/\sigma^4$. Plain denoiser MSE would reweight noise levels.

Recap of Previous Lecture

What Does the Optimal Denoiser Predict?

Use uniform empirical data $\{\mathbf{x}_1, \ldots, \mathbf{x}_N\}$. For fixed $\mathbf{x}$ and $\sigma > 0$, let $a_i = \mathcal{N}(\mathbf{x}; \mathbf{x}_i, \sigma^2 \mathbf{I})$ be the likelihood of the noisy observation.

$$D^*(\mathbf{x}, \sigma) = \frac{\sum_{i=1}^{N} \exp\left(-\frac{\|\mathbf{x}-\mathbf{x}_i\|_2^2}{2\sigma^2}\right) \mathbf{x}_i}{\sum_{j=1}^{N} \exp\left(-\frac{\|\mathbf{x}-\mathbf{x}_j\|_2^2}{2\sigma^2}\right)} = \mathbb{E}[\mathbf{x}_0 \mid \mathbf{x}].$$

The exact formula scans **all N training examples at every sampling step**.

Sampling (Euler)

1. Choose $\sigma_0 > \sigma_1 > \cdots > \sigma_N = 0$ and sample $\mathbf{x}^{(0)} \sim \mathcal{N}(0, \sigma_0^2 \mathbf{I})$.
2. For $i = 0, \ldots, N-1$, evaluate $D_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}, \sigma_i)$ and update:

$$\mathbf{x}^{(i+1)} = \mathbf{x}^{(i)} + (\sigma_{i+1} - \sigma_i) \frac{\mathbf{x}^{(i)} - D_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}, \sigma_i)}{\sigma_i}.$$

3. Return $\mathbf{x}^{(N)}$.

Karras T., et al. Elucidating the Design Space of Diffusion-Based Generative Models, 2022 — Appendix B.3, Eqs. (53–57)
Karras T., et al. Elucidating the Design Space of Diffusion-Based Generative Models, 2022

Recap of Previous Lecture

DDIM Is Euler in Noise Coordinates

Recall deterministic DDIM ($s < t$):

$$\hat{\boldsymbol{\epsilon}} = \boldsymbol{\epsilon}_{\boldsymbol{\theta},t}(\mathbf{x}_t), \qquad \hat{\mathbf{x}}_0 = \frac{\mathbf{x}_t - \sqrt{1-\bar{\alpha}_t}\,\hat{\boldsymbol{\epsilon}}}{\sqrt{\bar{\alpha}_t}},$$

$$\mathbf{x}_s = \sqrt{\bar{\alpha}_s}\,\hat{\mathbf{x}}_0 + \sqrt{1-\bar{\alpha}_s}\,\hat{\boldsymbol{\epsilon}}.$$

Recall the DDPM / NCSN rescaling ($\sigma_s < \sigma_t$)

$$\tilde{\mathbf{x}}_u = \frac{\mathbf{x}_u}{\sqrt{\bar{\alpha}_u}}, \qquad \sigma_u = \sqrt{\frac{1-\bar{\alpha}_u}{\bar{\alpha}_u}}, \qquad u \in \{s,t\}.$$

$$D_{\boldsymbol{\theta}}(\tilde{\mathbf{x}}_t, \sigma_t) = \hat{\mathbf{x}}_0 = \tilde{\mathbf{x}}_t - \sigma_t \hat{\boldsymbol{\epsilon}}.$$

Rewrite DDIM as the denoiser Euler step

$$\begin{aligned}\tilde{\mathbf{x}}_s &= \hat{\mathbf{x}}_0 + \sigma_s \hat{\boldsymbol{\epsilon}} = \tilde{\mathbf{x}}_t + (\sigma_s - \sigma_t)\hat{\boldsymbol{\epsilon}} \\ &= \tilde{\mathbf{x}}_t + (\sigma_s - \sigma_t)\frac{\tilde{\mathbf{x}}_t - D_{\boldsymbol{\theta}}(\tilde{\mathbf{x}}_t, \sigma_t)}{\sigma_t}.\end{aligned}$$

DDIM implements this Euler step using the existing noise predictor.

Recap of Previous Lecture

Continuous-Time Normalizing Flows

Recall $\mathbf{x}_t = \mathbf{x}(t)$ for $t \in [0, 1]$:

$$\mathbf{x}_0 \sim p_0(\mathbf{x}) = p(\mathbf{x}), \quad \mathbf{x}_1 \sim p_1(\mathbf{x}) = p_{\text{data}}(\mathbf{x})$$

Time convention: unlike the diffusion sections, here $t = 0$ denotes noise and $t = 1$ denotes data. Sampling runs forward in time.

$$\frac{d\mathbf{x}_t}{dt} = \mathbf{v}(\mathbf{x}_t, t), \quad \text{with initial condition } \mathbf{x}(0) = \mathbf{x}_0.$$

The velocity $\mathbf{v}$ determines a flow $\boldsymbol{\psi}_t$: $\mathbf{x}_t = \boldsymbol{\psi}_t(\mathbf{x}_0)$.

KFP Theorem (Continuity Equation)

$$\frac{\partial p_t(\mathbf{x})}{\partial t} = -\operatorname{div}\left(\mathbf{v}(\mathbf{x}, t) p_t(\mathbf{x})\right) \Leftrightarrow \frac{d \log p_t(\mathbf{x}(t))}{dt} = -\operatorname{tr}\left(\frac{\partial \mathbf{v}(\mathbf{x}(t), t)}{\partial \mathbf{x}(t)}\right)$$

- Likelihood-based CNF training requires ODE solves and Jacobian trace evaluations.
- The adjoint method computes gradients through the ODE solution; repeated solves can be costly.

Recap of Previous Lecture

Flow Matching

$$\mathbb{E}_{t\sim U[0,1]}\mathbb{E}_{\mathbf{x}\sim p_t(\mathbf{x})}\left\|\mathbf{v}(\mathbf{x},t)-\mathbf{v}_{\boldsymbol{\theta}}(\mathbf{x},t)\right\|^2\to\min_{\boldsymbol{\theta}}$$

Flow matching fits velocities by regression, without model ODE solves or trace evaluations during training.

Training

1. Sample $t\sim U[0,1]$, $\mathbf{x}_t\sim p_t(\mathbf{x})$.
2. Compute loss $\mathcal{L}=\left\|\mathbf{v}(\mathbf{x}_t,t)-\mathbf{v}_{\boldsymbol{\theta}}(\mathbf{x}_t,t)\right\|^2$.

Sampling

1. Sample $\mathbf{x}_0\sim\mathcal{N}(0,\mathbf{I})$.
2. Solve the ODE to obtain $\mathbf{x}_1$:

$$\mathbf{x}_1=\boldsymbol{\psi}_1(\mathbf{x}_0)=\texttt{ODESolve}_v(\mathbf{x}_0,\boldsymbol{\theta},t_0=0,t_1=1).$$

Recap of Previous Lecture

Problem: The true vector field $\mathbf{v}(\mathbf{x}, t)$ is **unknown**.

Many velocity fields connect the same endpoints.

To build a tractable training target, introduce a latent variable $\mathbf{z}$ and choose conditional paths whose mixture is the desired path:

$$p_t(\mathbf{x}) = \int p_t(\mathbf{x}|\mathbf{z})p(\mathbf{z})d\mathbf{z}$$

Here, $p_t(\mathbf{x}|\mathbf{z})$ is a **conditional probability path**.

The conditional probability path $p_t(\mathbf{x}|\mathbf{z})$ satisfies the KFP theorem:

$$\frac{\partial p_t(\mathbf{x}|\mathbf{z})}{\partial t} = -\operatorname{div}\left(\mathbf{v}(\mathbf{x}, \mathbf{z}, t)p_t(\mathbf{x}|\mathbf{z})\right),$$

where $\mathbf{v}(\mathbf{x}, \mathbf{z}, t)$ is a **conditional vector field**:

$$\frac{d\mathbf{x}_t}{dt} = \mathbf{v}(\mathbf{x}_t, t) \quad \Rightarrow \quad \frac{d\mathbf{x}_t}{dt} = \mathbf{v}(\mathbf{x}_t, \mathbf{z}, t)$$

Lipman Y., et al. Flow Matching for Generative Modeling, 2022
Tong A., et al. Improving and Generalizing Flow-Based Generative Models with Minibatch Optimal Transport, 2023

Recap of Previous Lecture

Theorem

The following vector field generates the probability path $p_t(\mathbf{x})$:

$$\mathbf{v}(\mathbf{x}, t) = \mathbb{E}_{p_t(\mathbf{z}|\mathbf{x})}\mathbf{v}(\mathbf{x}, \mathbf{z}, t) = \int \mathbf{v}(\mathbf{x}, \mathbf{z}, t)\frac{p_t(\mathbf{x}|\mathbf{z})p(\mathbf{z})}{p_t(\mathbf{x})}d\mathbf{z}$$

Recap of Previous Lecture

Conditional Flow Matching (CFM)

$$\mathbb{E}_{t\sim U[0,1]}\mathbb{E}_{\mathbf{z}\sim p(\mathbf{z})}\mathbb{E}_{\mathbf{x}\sim p_t(\mathbf{x}|\mathbf{z})}\left\|\mathbf{v}(\mathbf{x},\mathbf{z},t)-\mathbf{v}_{\boldsymbol{\theta}}(\mathbf{x},t)\right\|^2\to\min_{\boldsymbol{\theta}}$$

If $p_t(\mathbf{x})>0$ for all $\mathbf{x}\in\mathbb{R}^m$ and $0\leq t<1$, the FM and CFM objectives differ by a constant independent of $\boldsymbol{\theta}$, so they have the same minimizers.

Tong A., et al. Improving and Generalizing Flow-Based Generative Models with Minibatch Optimal Transport, 2023

Recap of Previous Lecture

Training

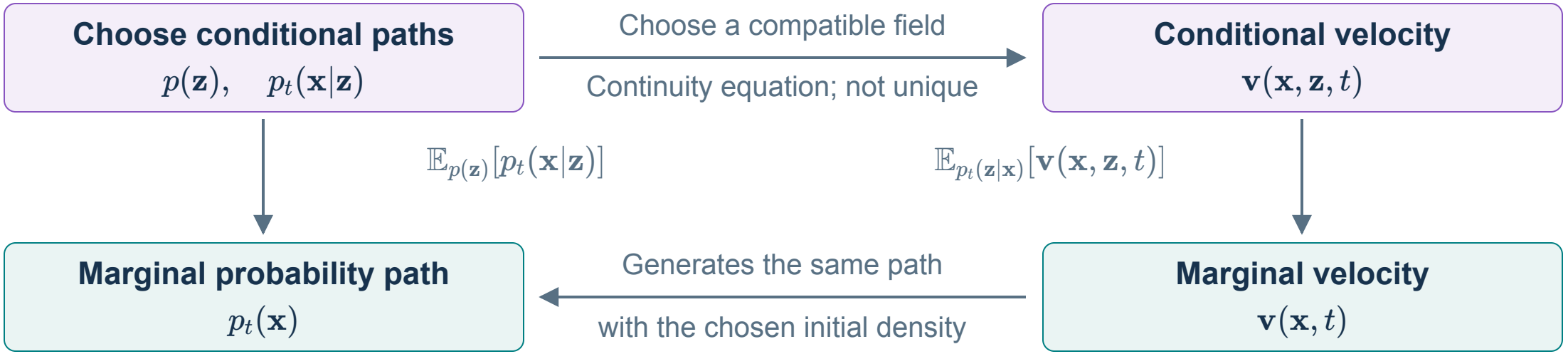
1. Sample $t \sim U[0, 1]$, $\mathbf{z} \sim p(\mathbf{z})$, $\mathbf{x}_t \sim p_t(\mathbf{x}|\mathbf{z})$.
2. Compute loss $\mathcal{L} = \|\mathbf{v}(\mathbf{x}_t, \mathbf{z}, t) - \mathbf{v}_{\boldsymbol{\theta}}(\mathbf{x}_t, t)\|^2$.

Sampling

1. Sample $\mathbf{x}_0 \sim \mathcal{N}(0, \mathbf{I})$.
2. Solve the ODE to obtain $\mathbf{x}_1$:

$$\mathbf{x}_1 = \boldsymbol{\psi}_1(\mathbf{x}_0) = \texttt{ODESolve}_v(\mathbf{x}_0, \boldsymbol{\theta}, t_0 = 0, t_1 = 1).$$

Recap of Previous Lecture



Open Questions

- **Q1** How should we choose the convenient conditioning latent variable $\mathbf{z}$?
- **Q2** How should we choose $p_t(\mathbf{x}|\mathbf{z})$ so that its marginal satisfies the endpoint constraints?

$$\mathbb{E}_{p(\mathbf{z})}p_0(\mathbf{x}|\mathbf{z}) = \mathcal{N}(0,\mathbf{I}); \quad \mathbb{E}_{p(\mathbf{z})}p_1(\mathbf{x}|\mathbf{z}) = p_{\text{data}}(\mathbf{x}).$$

Diagram adapted from A Visual Dive into Conditional Flow Matching

Recap of Previous Lecture

Gaussian Conditional Probability Path

On the interval considered, assume $\boldsymbol{\mu}_t(\mathbf{z})$ and $\boldsymbol{\sigma}_t(\mathbf{z})$ are differentiable in t, with $\boldsymbol{\sigma}_t(\mathbf{z}) > 0$ componentwise.

$$p_t(\mathbf{x}|\mathbf{z}) = \mathcal{N}\left(\boldsymbol{\mu}_t(\mathbf{z}), \boldsymbol{\sigma}_t^2(\mathbf{z})\right);$$

$$\mathbf{x}_t = \boldsymbol{\psi}_t(\mathbf{x}_0, \mathbf{z}) = \boldsymbol{\mu}_t(\mathbf{z}) + \boldsymbol{\sigma}_t(\mathbf{z}) \odot \boldsymbol{\epsilon}$$

image credit: A Visual Dive into Conditional Flow Matching

Recap of Previous Lecture

Gaussian Conditional Probability Path

On the interval considered, assume $\boldsymbol{\mu}_t(\mathbf{z})$ and $\boldsymbol{\sigma}_t(\mathbf{z})$ are differentiable in t, with $\boldsymbol{\sigma}_t(\mathbf{z}) > 0$ componentwise.

$$p_t(\mathbf{x}|\mathbf{z}) = \mathcal{N}\left(\boldsymbol{\mu}_t(\mathbf{z}), \boldsymbol{\sigma}_t^2(\mathbf{z})\right);$$

$$\mathbf{x}_t = \boldsymbol{\psi}_t(\mathbf{x}_0, \mathbf{z}) = \boldsymbol{\mu}_t(\mathbf{z}) + \boldsymbol{\sigma}_t(\mathbf{z}) \odot \boldsymbol{\epsilon}$$

$$\begin{aligned}\mathbf{v}(\mathbf{x}_t, \mathbf{z}, t) &= \boldsymbol{\psi}_t'\left(\boldsymbol{\psi}_t(\mathbf{x}_t, \mathbf{z})^{-1}, \mathbf{z}\right) \\ &= \boldsymbol{\mu}_t'(\mathbf{z}) + \frac{\boldsymbol{\sigma}_t'(\mathbf{z})}{\boldsymbol{\sigma}_t(\mathbf{z})} \odot (\mathbf{x}_t - \boldsymbol{\mu}_t(\mathbf{z}))\end{aligned}$$

Recap of Previous Lecture

Conditioning Latent Variable [Q1]

Let us choose $\mathbf{z} = \mathbf{x}_1$. Then $p(\mathbf{z}) = p_1(\mathbf{x}_1)$.

$$p_t(\mathbf{x}) = \int p_t(\mathbf{x}|\mathbf{x}_1)p_1(\mathbf{x}_1)d\mathbf{x}_1$$

We need to ensure the boundary constraints:

$$\begin{cases} \mathbb{E}_{p(\mathbf{z})} p_0(\mathbf{x}|\mathbf{z}) = \mathcal{N}(0, \mathbf{I}) \\ \mathbb{E}_{p(\mathbf{z})} p_1(\mathbf{x}|\mathbf{z}) = p_{\text{data}}(\mathbf{x}) \end{cases} \Rightarrow \quad \begin{cases} p_0(\mathbf{x}|\mathbf{x}_1) = \mathcal{N}(0, \mathbf{I}); \\ p_1(\mathbf{x}|\mathbf{x}_1) = \delta(\mathbf{x} - \mathbf{x}_1) \end{cases}$$

Recap of Previous Lecture

$$p_0(\mathbf{x}|\mathbf{x}_1) = \mathcal{N}(0, \mathbf{I}); \quad p_1(\mathbf{x}|\mathbf{x}_1) = \delta(\mathbf{x} - \mathbf{x}_1)$$

Gaussian Conditional Probability Path

$$p_t(\mathbf{x}|\mathbf{x}_1) = \mathcal{N}\left(\boldsymbol{\mu}_t(\mathbf{x}_1), \boldsymbol{\sigma}_t^2(\mathbf{x}_1)\right);$$
$$\mathbf{x}_t = \boldsymbol{\mu}_t(\mathbf{x}_1) + \boldsymbol{\sigma}_t(\mathbf{x}_1) \odot \boldsymbol{\epsilon}$$

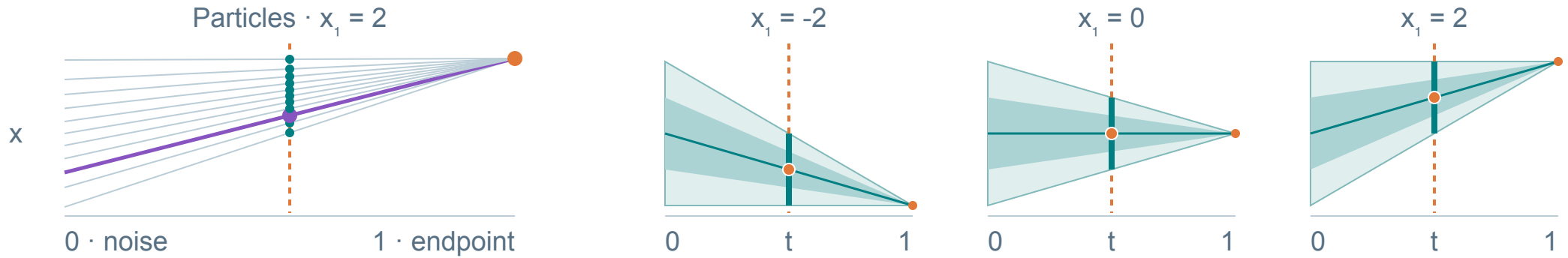
Let's consider straight conditional paths:

$$\begin{cases} \boldsymbol{\mu}_t(\mathbf{x}_1) = t\mathbf{x}_1 \\ \boldsymbol{\sigma}_t(\mathbf{x}_1) = 1 - t \end{cases} \Rightarrow \begin{cases} p_t(\mathbf{x}|\mathbf{x}_1) = \mathcal{N}\left(t\mathbf{x}_1, (1-t)^2 \cdot \mathbf{I}\right) \\ \mathbf{x}_t = t\mathbf{x}_1 + (1-t)\mathbf{x}_0 \end{cases}$$

image credit: A Visual Dive into Conditional Flow Matching

Recap of Previous Lecture

t = 0.5 · fixed particles; shaded bands: mean ± σ / ± 2σ.



Recap of Previous Lecture

Rectified Flow Objective

$$p_t(\mathbf{x}|\mathbf{x}_1) = \mathcal{N}\left(t\mathbf{x}_1, (1-t)^2\mathbf{I}\right); \quad \mathbf{x}_t = t\mathbf{x}_1 + (1-t)\mathbf{x}_0$$

$$\mathbf{v}(\mathbf{x}_t, \mathbf{x}_1, t) = \frac{\mathbf{x}_1 - \mathbf{x}_t}{1-t} = \mathbf{x}_1 - \mathbf{x}_0, \qquad t < 1$$

$$\mathbb{E}_{t\sim U[0,1]}\mathbb{E}_{\mathbf{z}\sim p(\mathbf{z})}\mathbb{E}_{\mathbf{x}\sim p_t(\mathbf{x}|\mathbf{z})} \left\|\mathbf{v}(\mathbf{x}, \mathbf{z}, t) - \mathbf{v}_{\boldsymbol{\theta}}(\mathbf{x}, t)\right\|^2$$

$$= \mathbb{E}_{t\sim U[0,1]}\mathbb{E}_{\mathbf{x}_1\sim p_{\text{data}}(\mathbf{x})}\mathbb{E}_{\mathbf{x}_0\sim\mathcal{N}(0,\mathbf{I})} \left\|(\mathbf{x}_1 - \mathbf{x}_0) - \mathbf{v}_{\boldsymbol{\theta}}\left(t\mathbf{x}_1 + (1-t)\mathbf{x}_0, t\right)\right\|^2$$

At $t = 1$, the quotient is undefined. Use the simplified training target $\mathbf{x}_1 - \mathbf{x}_0$, which contains no division.

image credit: https://mlg.eng.cam.ac.uk/blog/2024/01/20/flow-matching.html
Liu X., Gong C., Liu Q. Flow Straight and Fast: Learning to Generate and Transfer Data with Rectified Flow, 2022

Recap of Previous Lecture

- $\mathbf{v}(\mathbf{x}_t, \mathbf{x}_1, t)$ defines straight lines between $p_{\text{data}}(\mathbf{x})$ and $\mathcal{N}(0, \mathbf{I})$.
- **Straight training paths do not imply straight sampling trajectories.**



t = 0.5 · Same initial particles; endpoint pairings can differ.

Lipman Y., et al. Flow Matching for Generative Modeling, 2022
Liu X., Gong C., Liu Q. Flow Straight and Fast: Learning to Generate and Transfer Data with Rectified Flow, 2022

Recap of Previous Lecture

$$\mathbb{E}_{t\sim U[0,1]}\mathbb{E}_{\mathbf{x}_1\sim p_{\text{data}}(\mathbf{x})}\mathbb{E}_{\mathbf{x}_0\sim\mathcal{N}(0,\mathbf{I})}\left\|(\mathbf{x}_1-\mathbf{x}_0)-\mathbf{v}_{\boldsymbol{\theta}}(\mathbf{x}_t,t)\right\|^2\to\min_{\boldsymbol{\theta}}$$

Training

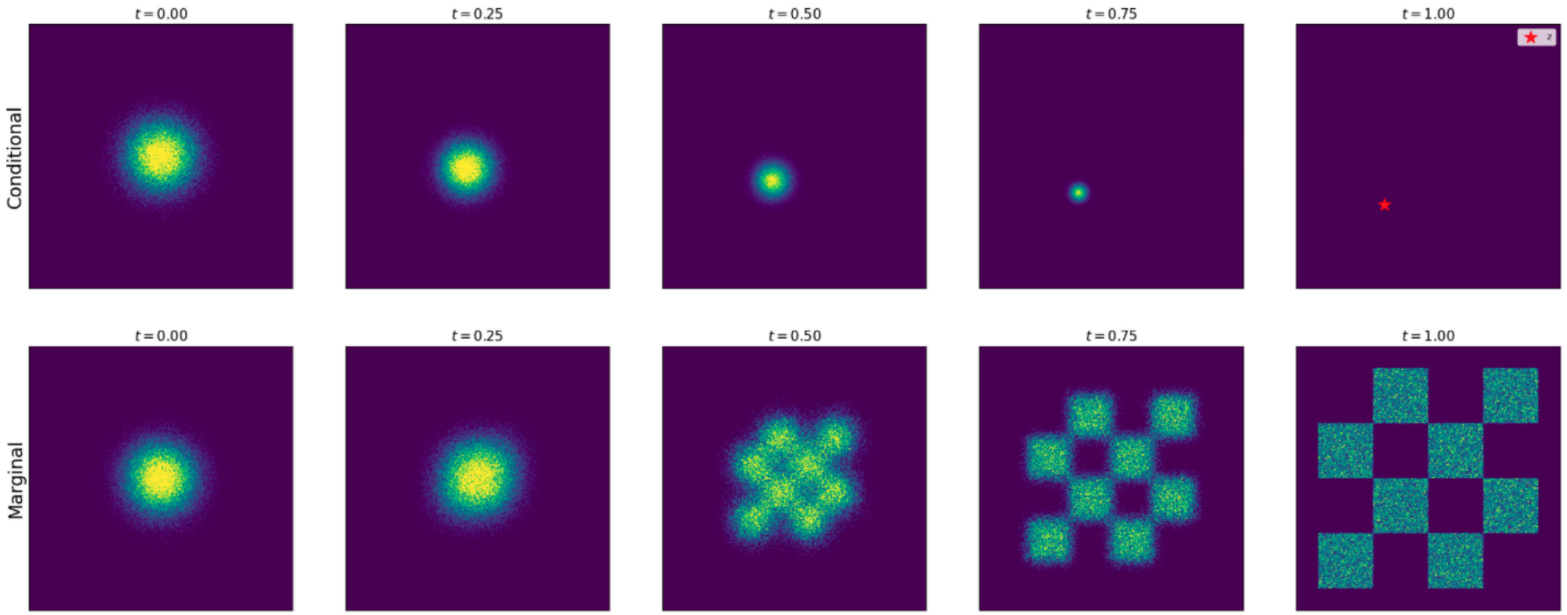
1. Sample $\mathbf{x}_1\sim p_{\text{data}}(\mathbf{x})$, $\mathbf{x}_0\sim\mathcal{N}(0,\mathbf{I})$, $t\sim U[0,1]$.
2. Compute noisy image $\mathbf{x}_t=t\mathbf{x}_1+(1-t)\mathbf{x}_0$.
3. Compute loss $\mathcal{L}=\left\|(\mathbf{x}_1-\mathbf{x}_0)-\mathbf{v}_{\boldsymbol{\theta}}(\mathbf{x}_t,t)\right\|^2$.

Sampling

1. Sample $\mathbf{x}_0\sim\mathcal{N}(0,\mathbf{I})$.
2. Solve the ODE to obtain $\mathbf{x}_1$:

$$\mathbf{x}_1=\boldsymbol{\psi}_1(\mathbf{x}_0)=\texttt{ODESolve}_v(\mathbf{x}_0,\boldsymbol{\theta},t_0=0,t_1=1).$$

Recap of Previous Lecture



- One-sided conditioning gives us the way to construct a generative model.
- Now we extend it to image-to-image formulation (mapping between two distinct data distributions $p_0(\mathbf{x})$ and $p_1(\mathbf{x})$).

Recap of Previous Lecture

Rectified Flow: Training-Time Distribution

Keep $\mathbf{x}_t = (1-t)\mathbf{x}_0 + t\mathbf{x}_1$ and the target $\mathbf{x}_1 - \mathbf{x}_0$ fixed. Let $\ell_{\boldsymbol{\theta}}(t)$ be the expected velocity MSE at time t.

$$\mathbb{E}_{t\sim r}\ell_{\boldsymbol{\theta}}(t) = \mathbb{E}_{t\sim U[0,1]}\left[r(t)\ell_{\boldsymbol{\theta}}(t)\right].$$

Changing the time distribution without correction changes the objective. To preserve uniform-time training, use $\ell_{\boldsymbol{\theta}}(t)/r(t)$ with $t \sim r$.

Logit-Normal Example

$$u \sim \mathcal{N}(\mu, s^2), \qquad t = \frac{1}{1+e^{-u}}, \qquad \log \mathrm{SNR}(t) = 2\log\frac{t}{1-t} = 2u.$$

Time $t=0$ is noise and $t=1$ is data. This choice does not prescribe the ODE grid.

Recap of Previous Lecture

Classifier-Free Guidance for Velocity Fields

Here $\mathbf{y}$ is a label or text prompt, distinct from the CFM latent $\mathbf{z}$.

$$\mathbf{v}_{\boldsymbol{\theta}}^{\gamma}(\mathbf{x}, t, \mathbf{y}) = \mathbf{v}_{\boldsymbol{\theta}}(\mathbf{x}, t, \varnothing) + \gamma \left[\mathbf{v}_{\boldsymbol{\theta}}(\mathbf{x}, t, \mathbf{y}) - \mathbf{v}_{\boldsymbol{\theta}}(\mathbf{x}, t, \varnothing)\right].$$

$\gamma = 0$: unconditional; $\gamma = 1$: conditional; $\gamma > 1$: extrapolation.

Training

1. Sample paired data $(\mathbf{x}_1, \mathbf{y}) \sim p_{\text{data}}$ and form the rectified-flow sample $\mathbf{x}_t$ as before.
2. Set $\tilde{\mathbf{y}} = \varnothing$ with fixed probability; otherwise set $\tilde{\mathbf{y}} = \mathbf{y}$.
3. Minimize the same velocity loss:

$$\mathcal{L} = \left\| \mathbf{v}_{\boldsymbol{\theta}}(\mathbf{x}_t, t, \tilde{\mathbf{y}}) - (\mathbf{x}_1 - \mathbf{x}_0) \right\|^2.$$

Sampling

1. Sample $\mathbf{x}_0 \sim \mathcal{N}(0, \mathbf{I})$ and choose $\mathbf{y}, \gamma$.
2. Evaluate both predictions to form $\mathbf{v}_{\boldsymbol{\theta}}^{\gamma}$ at each solver evaluation.
3. Solve the guided ODE from $t = 0$ to $t = 1$:

$$\frac{d\mathbf{x}_t}{dt} = \mathbf{v}_{\boldsymbol{\theta}}^{\gamma}(\mathbf{x}_t, t, \mathbf{y}).$$

Ho J., Salimans T. Classifier-Free Diffusion Guidance, 2022
Esser P., et al. Scaling Rectified Flow Transformers for High-Resolution Image Synthesis, 2024

Outline

01 Two-Sided Conditioning

02 Link between Flow Matching and Score-Based Models

03 Discrete Diffusion Models

04 Forward Discrete Process

05 Reverse Diffusion Process

06 From Token To Sequence

Outline

01 Two-Sided Conditioning

02 Link between Flow Matching and Score-Based Models

03 Discrete Diffusion Models

04 Forward Discrete Process

05 Reverse Diffusion Process

06 From Token To Sequence

Pair Conditioning

Conditional Flow Matching

$$\mathbb{E}_{t\sim U[0,1]}\mathbb{E}_{\mathbf{z}\sim p(\mathbf{z})}\mathbb{E}_{\mathbf{x}\sim p_t(\mathbf{x}|\mathbf{z})}\left\|\mathbf{v}(\mathbf{x},\mathbf{z},t)-\mathbf{v}_{\boldsymbol{\theta}}(\mathbf{x},t)\right\|^2\to\min_{\boldsymbol{\theta}}$$

Conditioning Latent Variable [Q1]

Let us choose $\mathbf{z}=(\mathbf{x}_0,\mathbf{x}_1)$. Start with independent endpoints: $p(\mathbf{z})=p_0(\mathbf{x}_0)p_1(\mathbf{x}_1)$.

$$p_t(\mathbf{x})=\int p_t(\mathbf{x}|\mathbf{x}_0,\mathbf{x}_1)p_0(\mathbf{x}_0)p_1(\mathbf{x}_1)d\mathbf{x}_0d\mathbf{x}_1$$

Pair Conditioning

We must enforce boundary constraints:

$$\begin{cases} p_0(\mathbf{x}) = \mathbb{E}_{p(\mathbf{z})} p_0(\mathbf{x}|\mathbf{z}); \\ p_1(\mathbf{x}) = \mathbb{E}_{p(\mathbf{z})} p_1(\mathbf{x}|\mathbf{z}) \end{cases} \Leftarrow \quad \begin{cases} p_0(\mathbf{x}|\mathbf{x}_0, \mathbf{x}_1) = \delta(\mathbf{x} - \mathbf{x}_0) \\ p_1(\mathbf{x}|\mathbf{x}_0, \mathbf{x}_1) = \delta(\mathbf{x} - \mathbf{x}_1) \end{cases}$$

Choosing these Dirac endpoints is sufficient to satisfy the boundary constraints. Gaussian smoothing recovers them in the zero-variance limit.

Two-Sided Conditioning

Desired conditional endpoints:

$$p_0(\mathbf{x}|\mathbf{x}_0, \mathbf{x}_1) = \delta(\mathbf{x} - \mathbf{x}_0); \quad p_1(\mathbf{x}|\mathbf{x}_0, \mathbf{x}_1) = \delta(\mathbf{x} - \mathbf{x}_1)$$

Gaussian Conditional Probability Path

$$p_t(\mathbf{x}|\mathbf{x}_0, \mathbf{x}_1) = \mathcal{N}\left(\boldsymbol{\mu}_t(\mathbf{x}_0, \mathbf{x}_1), \boldsymbol{\sigma}_t^2(\mathbf{x}_0, \mathbf{x}_1)\right);$$
$$\mathbf{x}_t = \boldsymbol{\mu}_t(\mathbf{x}_0, \mathbf{x}_1) + \boldsymbol{\sigma}_t(\mathbf{x}_0, \mathbf{x}_1) \odot \boldsymbol{\epsilon}$$

Here $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$. Positive Gaussian scales give smooth densities; the desired Dirac endpoints require the endpoint scales to vanish.

image credit: A Visual Dive into Conditional Flow Matching

Two-Sided Conditioning

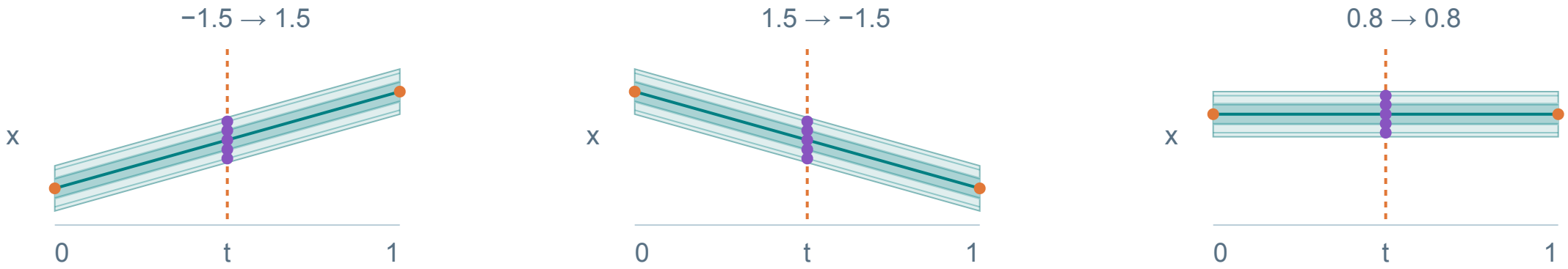
Straight conditional paths with constant Gaussian smoothing $\sigma > 0$:

$$\boldsymbol{\mu}_t = t\mathbf{x}_1 + (1-t)\mathbf{x}_0, \qquad p_t^\sigma(\mathbf{x}|\mathbf{x}_0, \mathbf{x}_1) = \mathcal{N}(\boldsymbol{\mu}_t, \sigma^2\mathbf{I}).$$

$$p_i^\sigma(\mathbf{x}|\mathbf{x}_0, \mathbf{x}_1) = \mathcal{N}(\mathbf{x}_i, \sigma^2\mathbf{I}) \xrightarrow[\sigma\to 0]{\text{weakly}} \delta(\mathbf{x} - \mathbf{x}_i), \qquad i \in \{0, 1\}.$$

For $\sigma > 0$, the marginal endpoints are Gaussian-smoothed versions of p_0 and p_1.

t = 0.5 · σ = 0.35 · bands: mean ± σ_t / ± $2\sigma_t$; center line: mean path.



Bands: mean ± σ_t / ± $2\sigma_t$. Fixed Gaussian quantiles; two-sided endpoints retain width σ.

Flow Matching: One-Sided vs. Two-Sided Conditioning

$$\mathbf{z} = \mathbf{x}_1$$
$$p_t(\mathbf{x}|\mathbf{x}_1) = \mathcal{N}\left(t\mathbf{x}_1, (1-t)^2\mathbf{I}\right)$$
$$\mathbf{x}_t = t\mathbf{x}_1 + (1-t)\mathbf{x}_0$$

$$\mathbf{z} = (\mathbf{x}_0, \mathbf{x}_1)$$
$$p_t^\sigma(\mathbf{x}|\mathbf{x}_0, \mathbf{x}_1) = \mathcal{N}\left(t\mathbf{x}_1 + (1-t)\mathbf{x}_0, \sigma^2\mathbf{I}\right)$$
$$\mathbf{x}_t = t\mathbf{x}_1 + (1-t)\mathbf{x}_0 + \sigma\boldsymbol{\epsilon}$$

$\sigma > 0$ is the smoothing scale; $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ is independent Gaussian noise.

t = 0.5 · σ = 0.35 · bands: mean ± σ_t / ± $2\sigma_t$; center line: mean path.



Bands: mean ± σ_t / ± $2\sigma_t$. Fixed Gaussian quantiles; two-sided endpoints retain width σ.

Two-Sided Conditioning: Vector Field and Objective

$$p_t^\sigma(\mathbf{x}|\mathbf{x}_0, \mathbf{x}_1) = \mathcal{N}\left(t\mathbf{x}_1 + (1-t)\mathbf{x}_0, \sigma^2\mathbf{I}\right), \qquad \sigma > 0;$$

$$\mathbf{x}_t = t\mathbf{x}_1 + (1-t)\mathbf{x}_0 + \sigma\boldsymbol{\epsilon}$$

Conditional Vector Field

Hold the sampled $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ fixed along the path.

$$\begin{aligned}\frac{d\mathbf{x}_t}{dt} &= \mathbf{v}(\mathbf{x}_t, \mathbf{x}_0, \mathbf{x}_1, t) \\ &= \boldsymbol{\mu}_t'(\mathbf{x}_0, \mathbf{x}_1) + \frac{\boldsymbol{\sigma}_t'(\mathbf{x}_0, \mathbf{x}_1)}{\boldsymbol{\sigma}_t(\mathbf{x}_0, \mathbf{x}_1)} \odot (\mathbf{x}_t - \boldsymbol{\mu}_t(\mathbf{x}_0, \mathbf{x}_1))\end{aligned}$$

$$\mathbf{v}(\mathbf{x}_t, \mathbf{x}_0, \mathbf{x}_1, t) = \mathbf{x}_1 - \mathbf{x}_0$$

For $\sigma > 0$, the scale is constant in time: $\boldsymbol{\sigma}_t' = \mathbf{0}$. The resulting target also applies in the deterministic limit.

Lipman Y. et al. Flow Matching for Generative Modeling, 2022.
Tong A. et al. Improving and Generalizing Flow-Based Generative Models with Minibatch Optimal Transport, 2023.

Two-Sided Conditioning: Vector Field and Objective

From now on, use the deterministic limit $\sigma = 0$:

$$\mathbf{x}_t = t\mathbf{x}_1 + (1-t)\mathbf{x}_0, \qquad p_t(\mathbf{x}|\mathbf{x}_0, \mathbf{x}_1) = \delta\big(\mathbf{x} - \mathbf{x}_t\big).$$

Conditional Flow Matching

$$\mathbb{E}_{t\sim U[0,1]}\mathbb{E}_{\mathbf{z}\sim p(\mathbf{z})}\mathbb{E}_{\mathbf{x}\sim p_t(\mathbf{x}|\mathbf{z})} \left\|\mathbf{v}(\mathbf{x}, \mathbf{z}, t) - \mathbf{v}_{\boldsymbol{\theta}}(\mathbf{x}, t)\right\|^2$$
$$= \mathbb{E}_{t\sim U[0,1]}\mathbb{E}_{(\mathbf{x}_0,\mathbf{x}_1)\sim p(\mathbf{x}_0,\mathbf{x}_1)} \left\|(\mathbf{x}_1 - \mathbf{x}_0) - \mathbf{v}_{\boldsymbol{\theta}}(\mathbf{x}_t, t)\right\|^2$$

- With independent $\mathbf{x}_0 \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$, this recovers the one-sided training procedure.
- Now, we do not require that $p_0(\mathbf{x})$ is necessarily $\mathcal{N}(0, \mathbf{I})$.

Lipman Y. et al. Flow Matching for Generative Modeling, 2022.
Tong A. et al. Improving and Generalizing Flow-Based Generative Models with Minibatch Optimal Transport, 2023.

CFM with Two-Sided Conditioning: Training and Sampling

- This conditioning allows us to transport any distribution $p_0(\mathbf{x})$ to any distribution $p_1(\mathbf{x})$.
- It's possible to apply this approach to paired tasks, e.g., style transfer.

Training

1. Sample $(\mathbf{x}_0, \mathbf{x}_1) \sim p(\mathbf{x}_0, \mathbf{x}_1)$, $t \sim U[0, 1]$.
2. Compute noisy image $\mathbf{x}_t = t\mathbf{x}_1 + (1-t)\mathbf{x}_0$.
3. Compute loss $\mathcal{L} = \|(\mathbf{x}_1 - \mathbf{x}_0) - \mathbf{v}_{\boldsymbol{\theta}}(\mathbf{x}_t, t)\|^2$.

CFM with Two-Sided Conditioning: Training and Sampling

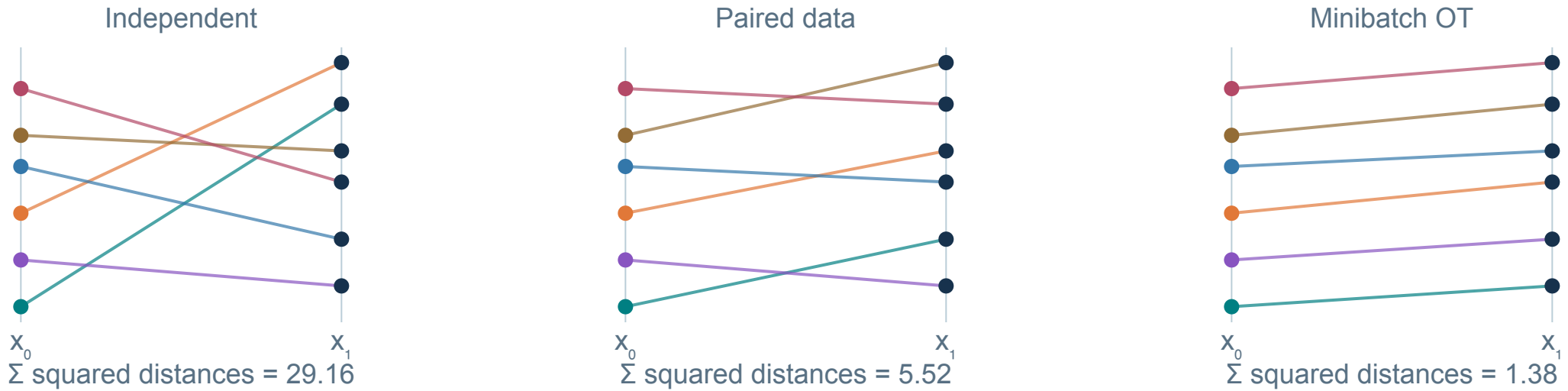
Sampling

1. Sample $\mathbf{x}_0 \sim p_0(\mathbf{x})$.
2. Solve the ODE to obtain $\mathbf{x}_1$:

$$\mathbf{x}_1 = \boldsymbol{\psi}_1(\mathbf{x}_0) = \texttt{ODESolve}_v(\mathbf{x}_0, \boldsymbol{\theta}, t_0 = 0, t_1 = 1).$$

Choosing Endpoint Pairs

A **coupling** $p(\mathbf{x}_0, \mathbf{x}_1)$ pairs endpoints while keeping marginals p_0 and p_1.



Six scalar endpoints per batch · paired data: illustrative correspondences · only the pairing changes.

$$\tau^* = \arg\min_{\tau \in S_B} \sum_{i=1}^{B} \left\| \mathbf{x}_0^{(i)} - \mathbf{x}_1^{(\tau(i))} \right\|_2^2 .$$

S_B: permutations of a batch of size B. Optimality is within each batch, not over the full distributions.

Same training rule: $\mathbf{x}_t = (1-t)\mathbf{x}_0 + t\mathbf{x}_1$, with velocity target $\mathbf{x}_1 - \mathbf{x}_0$.

Stable Diffusion 3: Scalable Flow Matching



- **Objective:** rectified flow with straight conditional paths and a velocity target.
- **Training times:** nonuniform sampling, including the logit-normal distribution.
- **Architecture:** MMDiT uses separate weights for text and image tokens, with joint attention between the two modalities.

Esser P., et al. Scaling Rectified Flow Transformers for High-Resolution Image Synthesis, 2024

Outline

01 Two-Sided Conditioning

02 Link between Flow Matching and Score-Based Models

03 Discrete Diffusion Models

04 Forward Discrete Process

05 Reverse Diffusion Process

06 From Token To Sequence

Score-Based Generative Models through SDEs

Training Objective

$$\mathbb{E}_{p_{\text{data}}(\mathbf{x}(0))}\mathbb{E}_{t\sim U[0,1]}\mathbb{E}_{q(\mathbf{x}(t)|\mathbf{x}(0))}\left\|\mathbf{s}_{\boldsymbol{\theta}}(\mathbf{x}(t),t)-\nabla_{\mathbf{x}(t)}\log q(\mathbf{x}(t)|\mathbf{x}(0))\right\|_2^2$$

Variance Exploding SDE (NCSN)

$$q(\mathbf{x}(t)|\mathbf{x}(0))=\mathcal{N}\left(\mathbf{x}(0),\left[\sigma^2(t)-\sigma^2(0)\right]\cdot\mathbf{I}\right),\quad \sigma(0)=0$$

Variance Preserving SDE (DDPM)

$$q(\mathbf{x}(t)|\mathbf{x}(0))=\mathcal{N}\left(\mathbf{x}(0)\alpha(t),\left(1-\alpha(t)^2\right)\cdot\mathbf{I}\right);$$
$$\alpha(t)=e^{-\frac{1}{2}\int_0^t\beta(s)ds}$$

Score-Based Generative Models through SDEs

Flow matching uses reverse time direction:

$$p_t(\mathbf{x}|\mathbf{x}_1) = q_{1-t}(\mathbf{x}|\mathbf{x}_0 = \mathbf{x}_1)$$

Tweedie's Formula

Recall Lecture 6, now with a scaled signal: for $\alpha_t > 0$, $\sigma_t > 0$,

$$p_t(\mathbf{x}_t|\mathbf{x}_1) = \mathcal{N}(\alpha_t \mathbf{x}_1, \sigma_t^2 \mathbf{I}) \quad \Rightarrow \quad \mathbb{E}[\mathbf{x}_1 \mid \mathbf{x}_t] = \frac{\mathbf{x}_t + \sigma_t^2 \nabla_{\mathbf{x}_t} \log p_t(\mathbf{x}_t)}{\alpha_t}.$$

When the conditional velocity is affine in $\mathbf{x}_1$, averaging over $\mathbf{x}_1 \mid \mathbf{x}_t$ expresses the marginal velocity through the score.

Lipman Y., et al. Flow Matching for Generative Modeling, 2022
Efron B. Tweedie's Formula and Selection Bias, 2011

Score-Based Generative Models through SDEs

$$p_t(\mathbf{x}|\mathbf{x}_1) = q_{1-t}(\mathbf{x}|\mathbf{x}_0 = \mathbf{x}_1)$$

$$\textbf{VE (NCSN):}\ p_t(\mathbf{x}|\mathbf{x}_1) = \mathcal{N}\left(\mathbf{x}_1, \sigma_{1-t}^2 \cdot \mathbf{I}\right)$$

$$\textbf{VP (DDPM):}\ p_t(\mathbf{x}|\mathbf{x}_1) = \mathcal{N}\left(\alpha_{1-t}\mathbf{x}_1, \left(1 - \alpha_{1-t}^2\right) \cdot \mathbf{I}\right)$$

Score-Based Generative Models through SDEs

Flow Matching Probability Path

$$p_t(\mathbf{x}|\mathbf{x}_1) = \mathcal{N}\left(t\mathbf{x}_1, (1-t)^2\mathbf{I}\right); \quad \mathbf{v}(\mathbf{x}_t, \mathbf{x}_1, t) = \frac{\mathbf{x}_1 - \mathbf{x}_t}{1-t}$$

$$\frac{d\mathbf{x}_t}{dt} = \mathbf{v}(\mathbf{x}_t, \mathbf{x}_1, t) = \boldsymbol{\mu}_t'(\mathbf{x}_1) + \frac{\boldsymbol{\sigma}_t'(\mathbf{x}_1)}{\boldsymbol{\sigma}_t(\mathbf{x}_1)} \odot (\mathbf{x}_t - \boldsymbol{\mu}_t(\mathbf{x}_1))$$

Let’s derive the conditional vector fields for VE (NCSN) and VP (DDPM).

Flow Matching vs Score-Based SDE Models

$$\frac{d\mathbf{x}_t}{dt} = \mathbf{v}(\mathbf{x}_t, \mathbf{x}_1, t) = \boldsymbol{\mu}'_t(\mathbf{x}_1) + \frac{\boldsymbol{\sigma}'_t(\mathbf{x}_1)}{\boldsymbol{\sigma}_t(\mathbf{x}_1)} \odot (\mathbf{x}_t - \boldsymbol{\mu}_t(\mathbf{x}_1))$$

Variance Exploding SDE Probability Path

$$p_t(\mathbf{x}|\mathbf{x}_1) = \mathcal{N}\left(\mathbf{x}_1, \sigma_{1-t}^2 \mathbf{I}\right) \quad \Rightarrow \quad \mathbf{v}(\mathbf{x}_t, \mathbf{x}_1, t) = -\frac{\sigma'_{1-t}}{\sigma_{1-t}}(\mathbf{x}_t - \mathbf{x}_1)$$

Variance Preserving SDE Probability Path

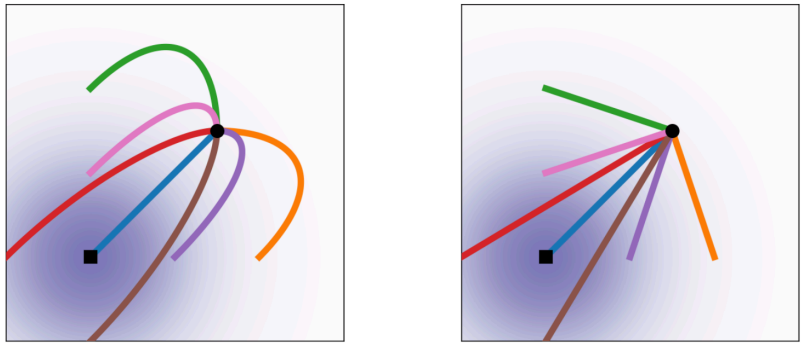
$$p_t(\mathbf{x}|\mathbf{x}_1) = \mathcal{N}\left(\alpha_{1-t}\mathbf{x}_1, (1 - \alpha_{1-t}^2)\mathbf{I}\right)$$

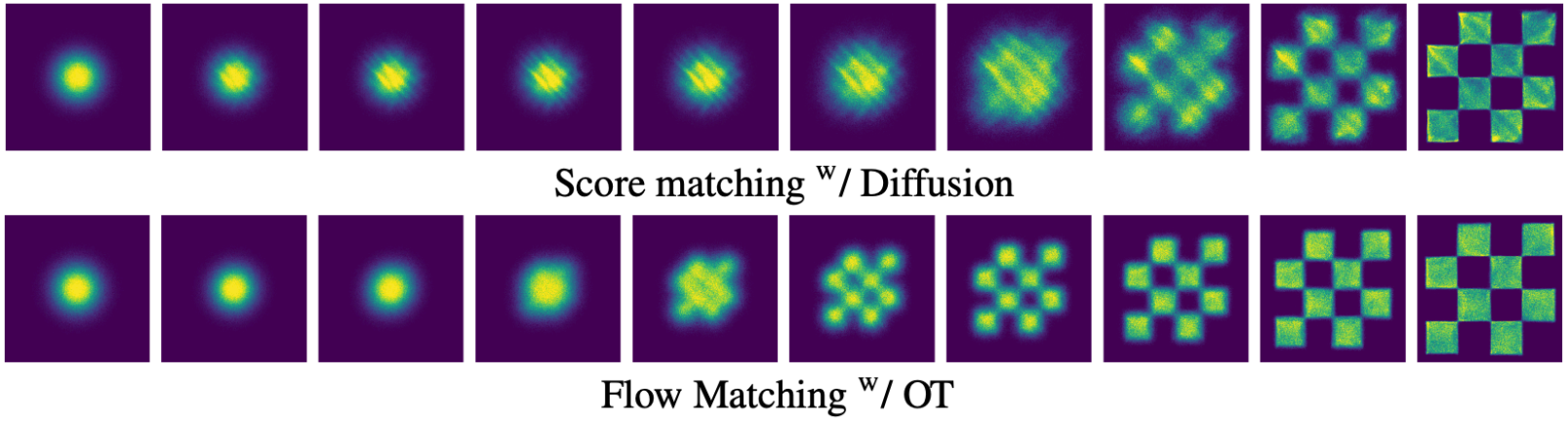
$$\Rightarrow \quad \mathbf{v}(\mathbf{x}_t, \mathbf{x}_1, t) = \frac{\alpha'_{1-t}}{1 - \alpha_{1-t}^2} \cdot (\alpha_{1-t}\mathbf{x}_t - \mathbf{x}_1)$$

Thus, VE/VP SDE models correspond to particular choices of the Gaussian probability path within the flow matching framework.

Flow Matching vs Score-Based SDE Models

Trajectories





Outline

01 Two-Sided Conditioning

02 Link between Flow Matching and Score-Based Models

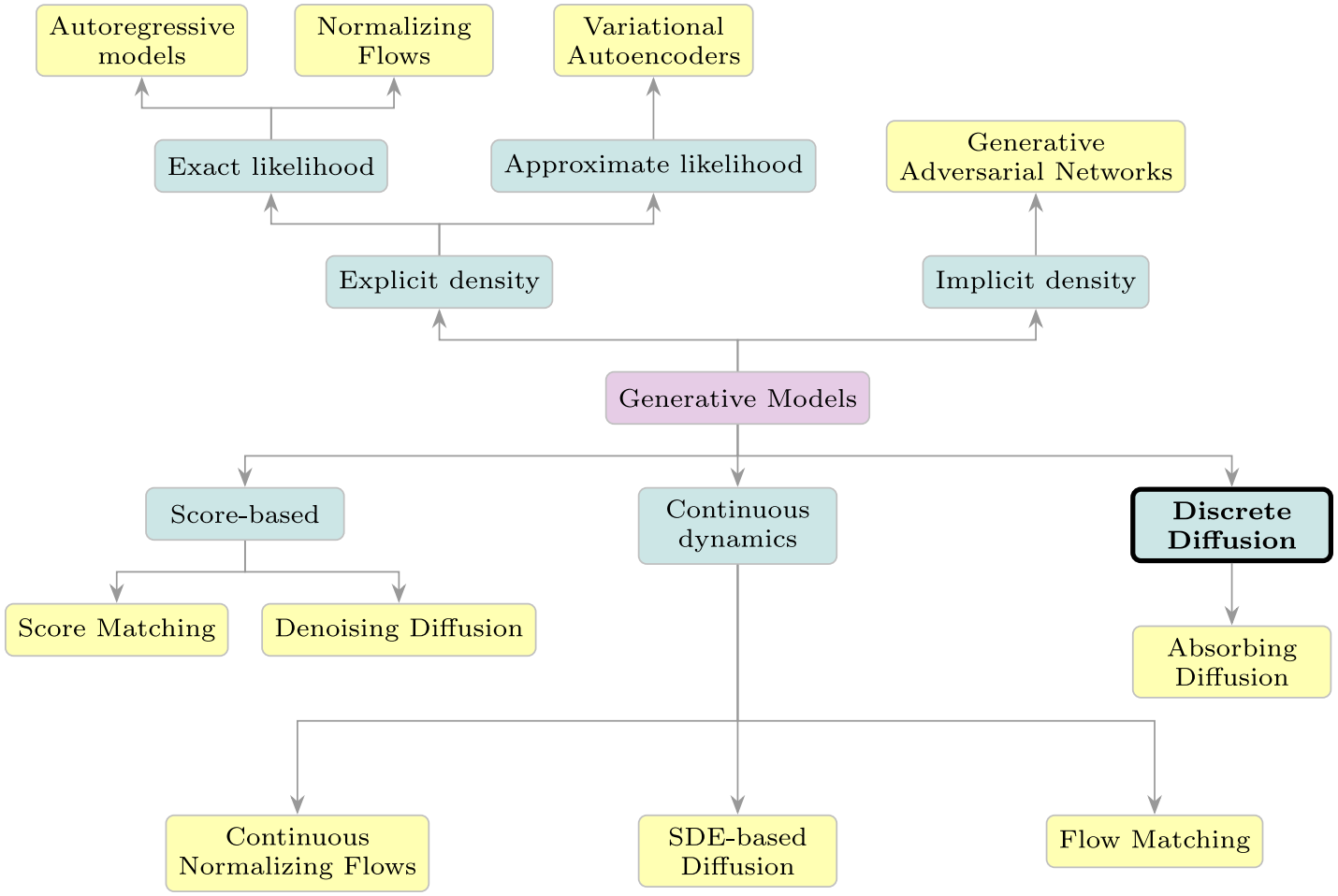
03 Discrete Diffusion Models

04 Forward Discrete Process

05 Reverse Diffusion Process

06 From Token To Sequence

Generative Models Taxonomy



Discrete or Continuous Diffusion Models?

Reminder: Diffusion models define a forward corruption process and a reverse denoising process. Previously, we studied diffusion models with continuous states $\mathbf{x}(t) \in \mathbb{R}^m$.

Continuous state space

- **Discrete time** $t \in \{0, 1, \dots, T\}$ $\Rightarrow$ **DDPM / NCSN**.
- **Continuous time** $t \in [0, 1]$ $\Rightarrow$ **Score-based SDE models**.

Now we turn to diffusion over discrete-value states $\mathbf{x}(t) \in \{1, \dots, K\}^m$.

Discrete state space

- **Discrete time** $t \in \{0, 1, \dots, T\}$.
- **Continuous time** $t \in [0, 1]$.

Let's discuss why we need discrete diffusion models.

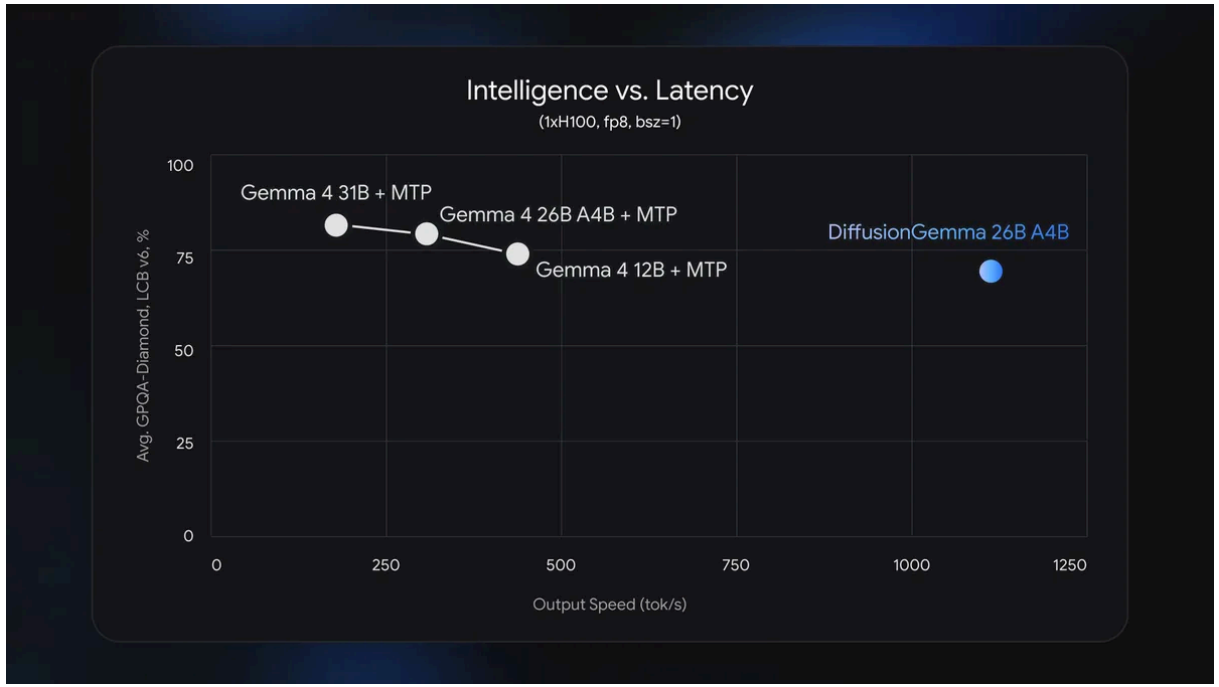
Why Discrete Diffusion Models?

Potential advantages over left-to-right autoregressive generation

- **Parallel generation:** generate multiple tokens per step, potentially reducing sequential decoding steps.
- **Bidirectional context:** use available context on both sides when predicting a token.
- **Flexible generation order:** choose which positions to generate next, including confidence-based ordering.
- **Infilling and controllability:** fill arbitrary gaps, preserve specified tokens, and steer sampling with guidance.
- **Iterative refinement:** revisit generated tokens when the sampler supports remasking or corrective updates.

Nie S. et al. Large language diffusion models, 2025. Wu C. et al. Fast-dLLM, 2025.
Schiff Y. et al. Simple guidance mechanisms for discrete diffusion models, 2025. Wang G. et al. Remasking discrete diffusion models with inference-time scaling, 2025.

Diffusion Language Models: DiffusionGemma



- **Parallel generation:** generate multiple tokens at each denoising step.
- **Practical speed:** over 1,000 output tokens per second on a single H100.

> How do we define corruption and denoising for discrete tokens?

1× H100 · FP8 · batch size 1.
O'Donoghue B., Flennerhag S. DiffusionGemma: 4x faster text generation, 2026.

Outline

01 Two-Sided Conditioning

02 Link between Flow Matching and Score-Based Models

03 Discrete Diffusion Models

04 Forward Discrete Process

05 Reverse Diffusion Process

06 From Token To Sequence

Forward Discrete Process

Continuous Diffusion Markov Chain

In continuous diffusion, the forward Markov chain is defined by progressively corrupting data with Gaussian noise:

$$q(\mathbf{x}_t|\mathbf{x}_{t-1}) = \mathcal{N}(\sqrt{1-\beta_t}\mathbf{x}_{t-1}, \beta_t\mathbf{I}).$$

Discrete Diffusion Markov Chain

For discrete data, we instead define a Markov chain over categorical states:

$$q(\mathbf{x}_t|\mathbf{x}_{t-1}) = \mathrm{Cat}(\mathbf{Q}_t\mathbf{x}_{t-1}),$$

- Each $\mathbf{x}_t \in \{0,1\}^K$ is a **one-hot vector** encoding the categorical state (it is just one token).
- What is the transition matrix $\mathbf{Q}_t$?

Austin J. et al. Structured denoising diffusion models in discrete state-spaces, 2021.

Forward Process over Time

Transition Matrix

$\mathbf{Q}_t \in [0,1]^{K \times K}$ is a **transition matrix** where each column gives transition probabilities from one state to all others, and columns sum to 1:

$$[\mathbf{Q}_t]_{ij} = q(x_t = i | x_{t-1} = j), \qquad \sum_{i=1}^{K} [\mathbf{Q}_t]_{ij} = 1.$$

- The forward diffusion gradually destroys information through repeated random transitions.
- Applying the transition t times yields the marginal distribution:

$$q(\mathbf{x}_t | \mathbf{x}_0) = \mathrm{Cat}(\mathbf{Q}_{1:t}\mathbf{x}_0), \qquad \mathbf{Q}_{1:t} = \mathbf{Q}_t \mathbf{Q}_{t-1} \cdots \mathbf{Q}_1.$$

- As $t \to T$, the process drives the data toward a stationary distribution.
- We design the transition matrices $\mathbf{Q}_t$ to achieve this behavior.

Austin J. et al. Structured denoising diffusion models in discrete state-spaces, 2021.

Transition Matrix

- The choice of $\mathbf{Q}_t$ determines how information is erased and what the stationary distribution becomes.
- $\mathbf{Q}_t$ and $\mathbf{Q}_{1:t}$ should be easy to compute for each t.

Common choices

- **Uniform diffusion**

  $$\mathbf{Q}_t = (1 - \beta_t)\mathbf{I} + \beta_t \mathbf{U}, \qquad \mathbf{U}_{ij} = \frac{1}{K}.$$

  Each token is replaced by a uniformly random symbol with probability β_t. The stationary distribution is uniform noise.

- **Absorbing diffusion**

  $$\mathbf{Q}_t = (1 - \beta_t)\mathbf{I} + \beta_t \, \mathbf{e}_m \mathbf{1}^\top.$$

  Tokens are gradually replaced by a special mask m; the stationary distribution is fully masked.

Austin J. et al. Structured denoising diffusion models in discrete state-spaces, 2021.

Transition Matrix

$$q(\mathbf{x}_t|\mathbf{x}_0) = \mathrm{Cat}(\mathbf{Q}_{1:t}\mathbf{x}_0).$$

Uniform Diffusion

$$\mathbf{Q}_t = (1-\beta_t)\mathbf{I} + \beta_t\mathbf{U}, \quad \mathbf{U}_{ij} = \tfrac{1}{K}; \qquad \mathbf{Q}_{1:t} = \bar{\alpha}_t\mathbf{I} + (1-\bar{\alpha}_t)\mathbf{U}, \quad \bar{\alpha}_t = \prod_{s=1}^{t}(1-\beta_s).$$

Clean token A · β = 0.25 · K = 4 · matrix at t = 4; distributions at t = 0, 4, 20.

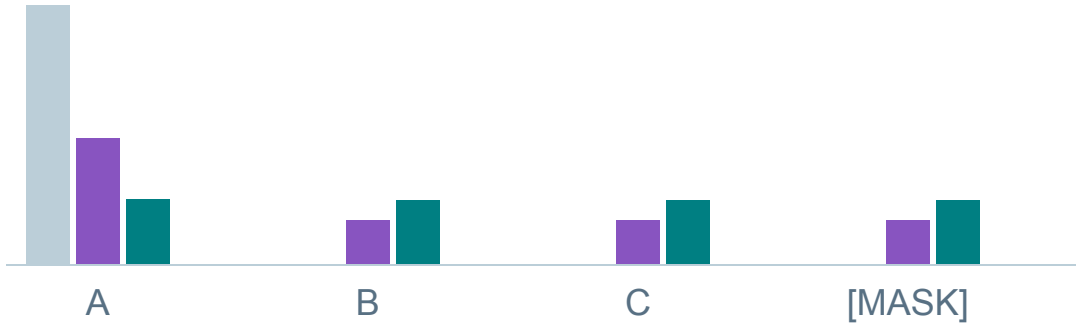
$Q_{1:t}$ · columns: initial state

	A	B	C	[MASK]
A	0.49	0.17	0.17	0.17
B	0.17	0.49	0.17	0.17
C	0.17	0.17	0.49	0.17
[MASK]	0.17	0.17	0.17	0.49

Highlighted column = $q(x_t \mid x_0)$

Limit: 0.25 per state, including the original token.

Probability · 0 → 4 → 20 steps



Gray: 0 · purple: 4 · teal: 20

Austin J. et al. Structured denoising diffusion models in discrete state-spaces, 2021.

Transition Matrix

Absorbing Diffusion

$$\mathbf{Q}_t = (1-\beta_t)\mathbf{I} + \beta_t\,\mathbf{e}_m\mathbf{1}^\top, \qquad \mathbf{Q}_{1:t} = \bar{\alpha}_t\mathbf{I} + (1-\bar{\alpha}_t)\mathbf{e}_m\mathbf{1}^\top, \quad \bar{\alpha}_t = \prod_{s=1}^{t}(1-\beta_s).$$

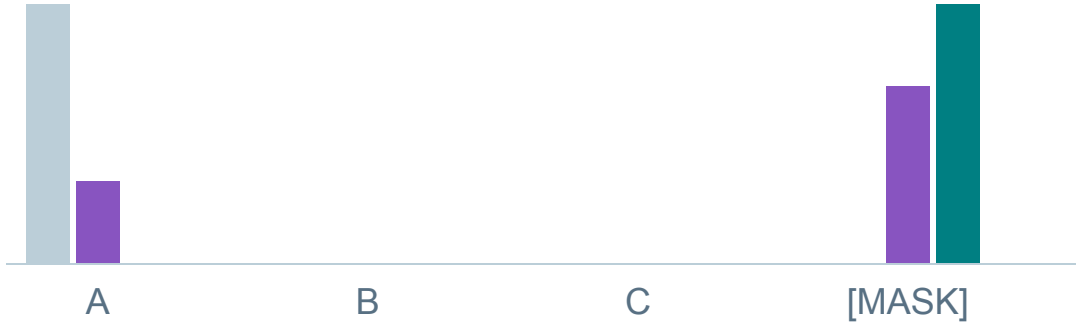
Clean token A · β = 0.25 · K = 4 · matrix at t = 4; distributions at t = 0, 4, 20.

$Q_{1:t}$ · columns: initial state

	A	B	C	[MASK]
A	0.32	0.00	0.00	0.00
B	0.00	0.32	0.00	0.00
C	0.00	0.00	0.32	0.00
[MASK]	0.68	0.68	0.68	1.00

Highlighted column = $q(x_t \mid x_0)$

Probability · 0 → 4 → 20 steps



Gray: 0 · purple: 4 · teal: 20

Limit: all probability at [MASK]; once masked, a sampled token stays masked.

As $\bar{\alpha}_t \to 0$, all probability moves to the mask: $q(\mathbf{x}_T) \approx \mathrm{Cat}(\mathbf{e}_m)$. This connects diffusion to **masked language modeling**.

Uniform vs. Absorbing Transition Matrix

Aspect	Uniform Diffusion	Absorbing Diffusion
$\mathbf{Q}_t$	$(1-\beta_t)\mathbf{I} + \beta_t \mathbf{U}$	$(1-\beta_t)\mathbf{I} + \beta_t \mathbf{e}_m \mathbf{1}^\top$
$\mathbf{Q}_{1:t}$	$\bar{\alpha}_t \mathbf{I} + (1-\bar{\alpha}_t)\mathbf{U}$	$\bar{\alpha}_t \mathbf{I} + (1-\bar{\alpha}_t)\mathbf{e}_m \mathbf{1}^\top$
$\mathbf{Q}_{1:\infty}$, $\bar{\alpha}_t \to 0$	$\mathbf{U}$	$\mathbf{e}_m \mathbf{1}^\top$
Interpretation	Random replacement	Gradual masking of tokens
Application	Image diffusion	Text diffusion $\approx$ Masked LM

For absorbing diffusion, the limiting token distribution is $\mathrm{Cat}(\mathbf{e}_m)$.

Observation

Both schemes gradually destroy information, but differ in their stationary limit. Absorbing diffusion bridges diffusion and masked-language-model objectives.

Austin J. et al. Structured denoising diffusion models in discrete state-spaces, 2021.

Outline

01 Two-Sided Conditioning

02 Link between Flow Matching and Score-Based Models

03 Discrete Diffusion Models

04 Forward Discrete Process

05 Reverse Diffusion Process

06 From Token To Sequence

Posterior of the Forward Process

ELBO

$$\mathcal{L}_{\phi,\theta}(\mathbf{x}) = \mathbb{E}_{q(\mathbf{x}_1|\mathbf{x}_0)} \log p_{\theta}(\mathbf{x}_0|\mathbf{x}_1) - D_{\mathrm{KL}}\big(q(\mathbf{x}_T|\mathbf{x}_0)\|p(\mathbf{x}_T)\big) - \sum_{t=2}^{T} \underbrace{\mathbb{E}_{q(\mathbf{x}_t|\mathbf{x}_0)} D_{\mathrm{KL}}\big(q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0)\|p_{\theta}(\mathbf{x}_{t-1}|\mathbf{x}_t)\big)}_{\mathcal{L}_t}$$

- Conditioned reverse distribution $q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0)$ played crucial role in the continuous-state diffusion model.
- It shows the probability of a previous state given the noisy state $\mathbf{x}_t$ and the original clean data $\mathbf{x}_0$.

Posterior of the Forward Process

Discrete conditioned reverse distribution

$$q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) = \frac{q(\mathbf{x}_t|\mathbf{x}_{t-1}, \mathbf{x}_0)\, q(\mathbf{x}_{t-1}|\mathbf{x}_0)}{q(\mathbf{x}_t|\mathbf{x}_0)}$$
$$= \frac{\mathrm{Cat}(\mathbf{Q}_t) \cdot \mathrm{Cat}(\mathbf{Q}_{1:t-1})}{\mathrm{Cat}(\mathbf{Q}_{1:t})}.$$

Austin J. et al. Structured denoising diffusion models in discrete state-spaces, 2021.

Posterior of the Forward Process

Discrete conditioned reverse distribution

$$q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) = \text{Cat}\left(\frac{\mathbf{Q}_t^\top \mathbf{x}_t \odot \mathbf{Q}_{1:t-1}\mathbf{x}_0}{\mathbf{x}_t^\top \mathbf{Q}_{1:t}\mathbf{x}_0}\right).$$

Recall the ELBO term

$$\mathcal{L}_t = \mathbb{E}_{q(\mathbf{x}_t|\mathbf{x}_0)} D_{\text{KL}}\big(q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) \,\|\, p_{\boldsymbol{\theta}}(\mathbf{x}_{t-1}|\mathbf{x}_t)\big),$$

- Both $q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0)$ and $q(\mathbf{x}_t|\mathbf{x}_0)$ are known analytically from the forward process.
- The reverse process $p_{\boldsymbol{\theta}}(\mathbf{x}_{t-1}|\mathbf{x}_t)$ is a learned categorical distribution:

$$p_{\boldsymbol{\theta}}(\mathbf{x}_{t-1}|\mathbf{x}_t) = \text{Cat}\big(\boldsymbol{\pi}_{\boldsymbol{\theta}}(\mathbf{x}_t, t)\big),$$

where $\boldsymbol{\pi}_{\boldsymbol{\theta}}$ is a neural network.

Discrete-time ELBO for Discrete Diffusion

ELBO term

$$\mathcal{L}_t = \mathbb{E}_{q(\mathbf{x}_t|\mathbf{x}_0)} D_{\mathrm{KL}}\big(q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) \,\|\, p_{\boldsymbol{\theta}}(\mathbf{x}_{t-1}|\mathbf{x}_t)\big).$$

Categorical KL

$$D_{\mathrm{KL}}\big(\mathrm{Cat}(\mathbf{q}) \,\|\, \mathrm{Cat}(\mathbf{p})\big) = \sum_{k=1}^{K} q_k \log \frac{q_k}{p_k} = \mathrm{H}(\mathbf{q}, \mathbf{p}) - \mathrm{H}(\mathbf{q}),$$

- $\mathrm{H}\big(q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0)\big)$ is a constant w.r.t. $\boldsymbol{\theta}$.
- $\mathrm{H}(\mathbf{q}, \mathbf{p}) = -\sum_k q_k \log p_k$ is a **cross-entropy loss**.

Therefore, minimizing $\mathcal{L}_t$ w.r.t. $\boldsymbol{\theta}$ is equivalent to minimizing

$$\mathbb{E}_{q(\mathbf{x}_t|\mathbf{x}_0)} \mathrm{H}\Big(q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0),\, p_{\boldsymbol{\theta}}(\mathbf{x}_{t-1}|\mathbf{x}_t)\Big).$$

Austin J. et al. Structured denoising diffusion models in discrete state-spaces, 2021.

Outline

01 Two-Sided Conditioning

02 Link between Flow Matching and Score-Based Models

03 Discrete Diffusion Models

04 Forward Discrete Process

05 Reverse Diffusion Process

06 From Token To Sequence

From Token to Sequence

One-hot sequence representation

$$\mathbf{x}_t \in \{0,1\}^K \quad \Leftrightarrow \quad \mathbf{X}_t \in \{0,1\}^{K \times m}$$

Here $\mathbf{X}_t$ is a one-hot representation of a sequence of tokens.

Independent Token-wise Forward Process

$$q(\mathbf{X}_t|\mathbf{X}_{t-1}) = \prod_{i=1}^{m} q(\mathbf{x}_t^i|\mathbf{x}_{t-1}^i) = \mathrm{Cat}(\mathbf{Q}_t\mathbf{X}_{t-1})$$

- Each position i evolves according to its own Markov chain.
- Often the same transition matrix $\mathbf{Q}_t$ is shared across i.

Continuous Diffusion Analogy

- In Gaussian DDPMs with diagonal covariance, noise is independent per pixel.
- Structure is not in the noise; it is learned by the reverse model.

Austin J. et al. Structured denoising diffusion models in discrete state-spaces, 2021.

From Token to Sequence

$$q(\mathbf{X}_t|\mathbf{X}_{t-1}) = \prod_{i=1}^{m} q(\mathbf{x}_t^i|\mathbf{x}_{t-1}^i) = \mathrm{Cat}(\mathbf{Q}_t\mathbf{X}_{t-1})$$

$$q(\mathbf{X}_t|\mathbf{X}_0) = \mathrm{Cat}(\mathbf{Q}_{1:t}\mathbf{X}_0)$$

Conditioned Reverse Distribution

$$q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) = \mathrm{Cat}\left(\frac{\mathbf{Q}_t^\top \mathbf{x}_t \odot \mathbf{Q}_{1:t-1}\mathbf{x}_0}{\mathbf{x}_t^\top \mathbf{Q}_{1:t}\mathbf{x}_0}\right).$$

$$q(\mathbf{X}_{t-1}|\mathbf{X}_t, \mathbf{X}_0) = \prod_{i=1}^{m} q(\mathbf{x}_{t-1}^i|\mathbf{x}_t^i, \mathbf{x}_0^i).$$

- All distributions defined by the forward process are factorized.
- Dependence appears in the learned reverse model.

Austin J. et al. Structured denoising diffusion models in discrete state-spaces, 2021.

Reverse Model for Sequence

$$p_{\boldsymbol{\theta}}(\mathbf{X}_{t-1}|\mathbf{X}_t) = \prod_{i=1}^{m} p_{\boldsymbol{\theta}}(\mathbf{x}_{t-1}^i|\mathbf{X}_t).$$

- The output factorizes (parallel prediction across positions).
- Each factor conditions on the entire noisy sequence $\mathbf{X}_t$.
- This predicts tokens from a corrupted sequence; masking corruption connects it to **masked language modeling**.

Objective: $\mathcal{L}_t$ term

$$\begin{aligned} &D_{\mathrm{KL}}\left(q(\mathbf{X}_{t-1}|\mathbf{X}_t, \mathbf{X}_0) \,\|\, p_{\boldsymbol{\theta}}(\mathbf{X}_{t-1}|\mathbf{X}_t)\right) \\ &= \sum_{i=1}^{m} D_{\mathrm{KL}}\left(q(\mathbf{x}_{t-1}^i|\mathbf{x}_t^i, \mathbf{x}_0^i) \| p_{\boldsymbol{\theta}}(\mathbf{x}_{t-1}^i|\mathbf{X}_t)\right). \end{aligned}$$

Reverse Model for Sequence

Simplified denoising objective

Predict clean tokens from the corrupted sequence (time conditioning is implicit):

$$\mathcal{L} = \sum_{t=1}^{T} \sum_{i=1}^{m} \mathbb{E}_{q(\mathbf{x}_t|\mathbf{x}_0)} \Big[-\log p_{\boldsymbol{\theta}}(\mathbf{x}_0^i | \mathbf{X}_t) \Big].$$

This unweighted clean-token cross-entropy is a simplified training objective; it is not the general discrete ELBO.

Connection to absorbing diffusion

With a reverse model that preserves visible tokens and uses the forward-posterior unmasking probability, the ELBO yields a time-weighted loss over masked positions. We derive this next.

Austin et al. Structured denoising diffusion models, 2021. Sahoo et al. Simple and effective masked diffusion language models, 2024.

Summary

- Two-sided conditioning supports independent pairs, paired data and minibatch OT couplings. The coupling changes pair sampling while preserving the interpolation and velocity target.
- Diffusion and score-based models are special cases of the flow matching approach, but use curved trajectories.
- Discrete diffusion offers parallel generation and flexible conditioning; token revision requires a suitable sampler.
- Forward discrete diffusion process defines a Markov chain with discrete states; uniform and absorbing transitions make it tractable.
- The discrete ELBO contains categorical KL terms; their parameter-dependent parts are cross-entropies against the forward posterior.
- For sequences, the forward process factorizes over tokens. Denoisers predict all positions from the whole corrupted sequence; clean-token cross-entropy is a simplified objective.